

Usare Wireshark per Osservare l'Handshake a 3 Vie TCP

Data: 16 giugno 2026

1. Sintesi

In questa esercitazione è stato utilizzato Wireshark per catturare e analizzare il traffico di rete generato tra due host virtuali all'interno di un ambiente Mininet. L'obiettivo era osservare e comprendere il meccanismo dell'handshake a 3 vie del protocollo TCP, verificando i flag SYN, SYN-ACK e ACK nei pacchetti catturati. Nella parte finale è stato usato anche tcpdump per visualizzare gli stessi dati da riga di comando.

2. Introduzione

Il protocollo TCP (Transmission Control Protocol) utilizza un processo detto handshake a 3 vie per stabilire una connessione affidabile tra due host. Questo processo si compone di tre fasi: il client invia un pacchetto con il flag SYN attivo per richiedere la connessione, il server risponde con SYN-ACK per confermare la disponibilità, e infine il client invia un ACK per completare la connessione. Solo al termine di questi tre scambi la sessione TCP è considerata stabilita e può iniziare la trasmissione dei dati.

3. Ambiente di Laboratorio

L'esercitazione è stata svolta sulla macchina virtuale CyberOps Workstation, avviata tramite VirtualBox. All'interno della VM è stato usato Mininet per simulare una rete con più host virtuali. In particolare, H1 (10.0.0.11) ha svolto il ruolo di client e H4 (172.16.0.40) quello di server web. Su H4 era in esecuzione un server nginx, mentre su H1 sono stati utilizzati Firefox come browser, tcpdump per la cattura del traffico e Wireshark per l'analisi dei pacchetti.

4. Procedura

Parte 1 — Preparazione degli host

Dopo aver avviato la VM e aperto un terminale, è stato lanciato lo script che genera la topologia Mininet con il comando `sudo lab.support.files/scripts/cyberops_topo.py`. La rete è stata creata correttamente, con host H1, H2, H3, H4, lo switch S1 e il router R1.



Figura 1 — Topologia Mininet avviata correttamente.

Tramite il prompt di Mininet sono stati aperti i terminali degli host H1 e H4 con i comandi xterm H1 e xterm H4.

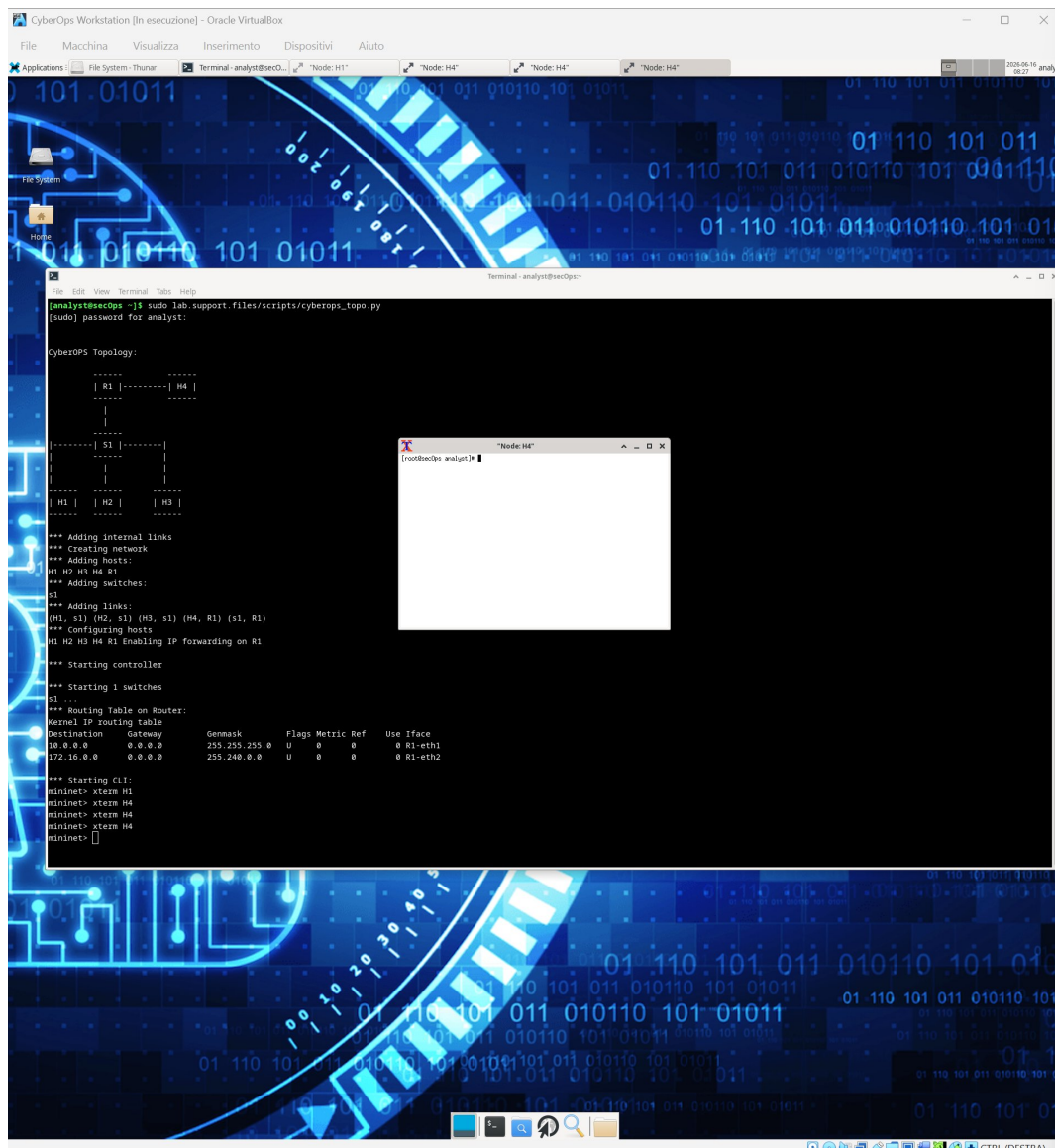


Figura 2 — Finestre xterm di H1 e H4 aperte.

Su H4 è stato avviato il server web nginx tramite lo script `reg_server_start.sh`. Su H1 è stato avviato Firefox come utente `analyst` e successivamente `tcpdump` per catturare 50 pacchetti salvandoli nel file `/home/analyst/capture.pcap`. Durante la cattura, Firefox ha navigato verso `172.16.0.40` confermando il corretto funzionamento del server.

Figura 4 — Lista dei pacchetti TCP filtrati in Wireshark.

Il pacchetto n. 19 rappresenta il primo passo dell'handshake. Ha come sorgente H1 (10.0.0.11) e destinazione H4 (172.16.0.40). Espandendo la sezione TCP si nota che il flag SYN è attivo (Flags: 0x002) mentre tutti gli altri, incluso ACK, risultano non impostati. Il numero di sequenza relativo è 0.

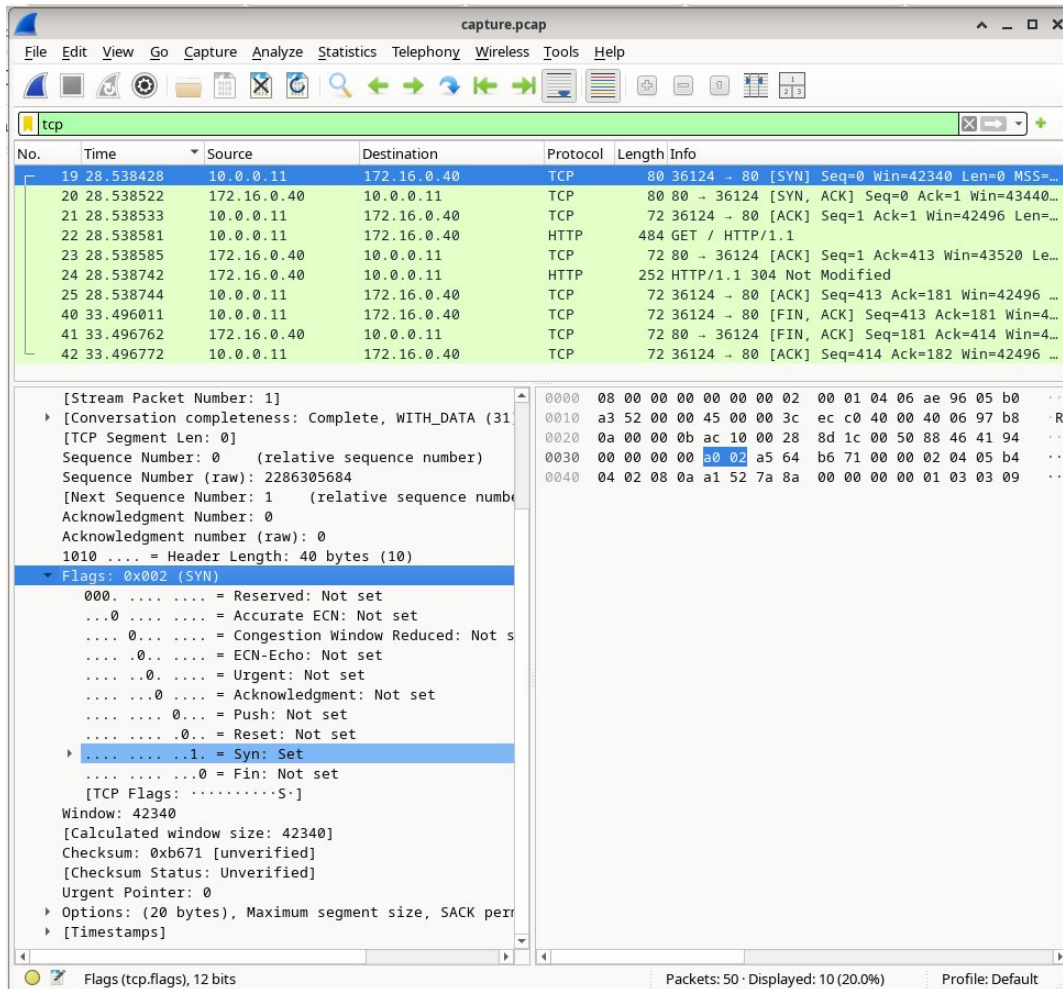


Figura 5 — Pacchetto SYN: flag 0x002, Syn: Set.

Il pacchetto n. 20 è la risposta di H4. In questo caso i flag SYN e ACK sono entrambi attivi (Flags: 0x012), il numero di sequenza relativo è 0 e l'acknowledgment number è 1.

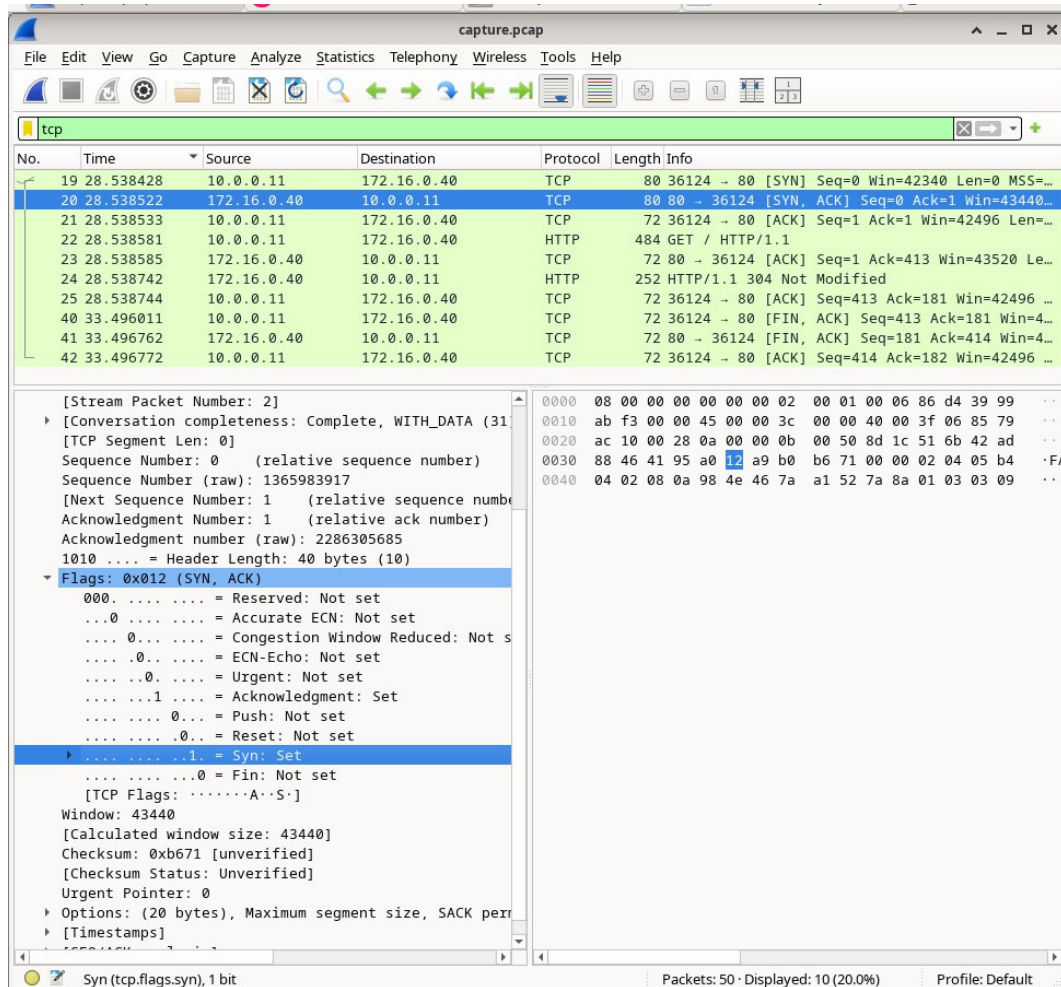


Figura 6 — Pacchetto SYN-ACK: flag 0x012, Syn e Acknowledgment: Set.

Il pacchetto n. 21 completa l'handshake. Proviene da H1 e ha attivo solo il flag ACK (Flags: 0x010). Sia il numero di sequenza che l'acknowledgment number sono impostati a 1. Da questo momento la connessione TCP è stabilita e la comunicazione può avere inizio.

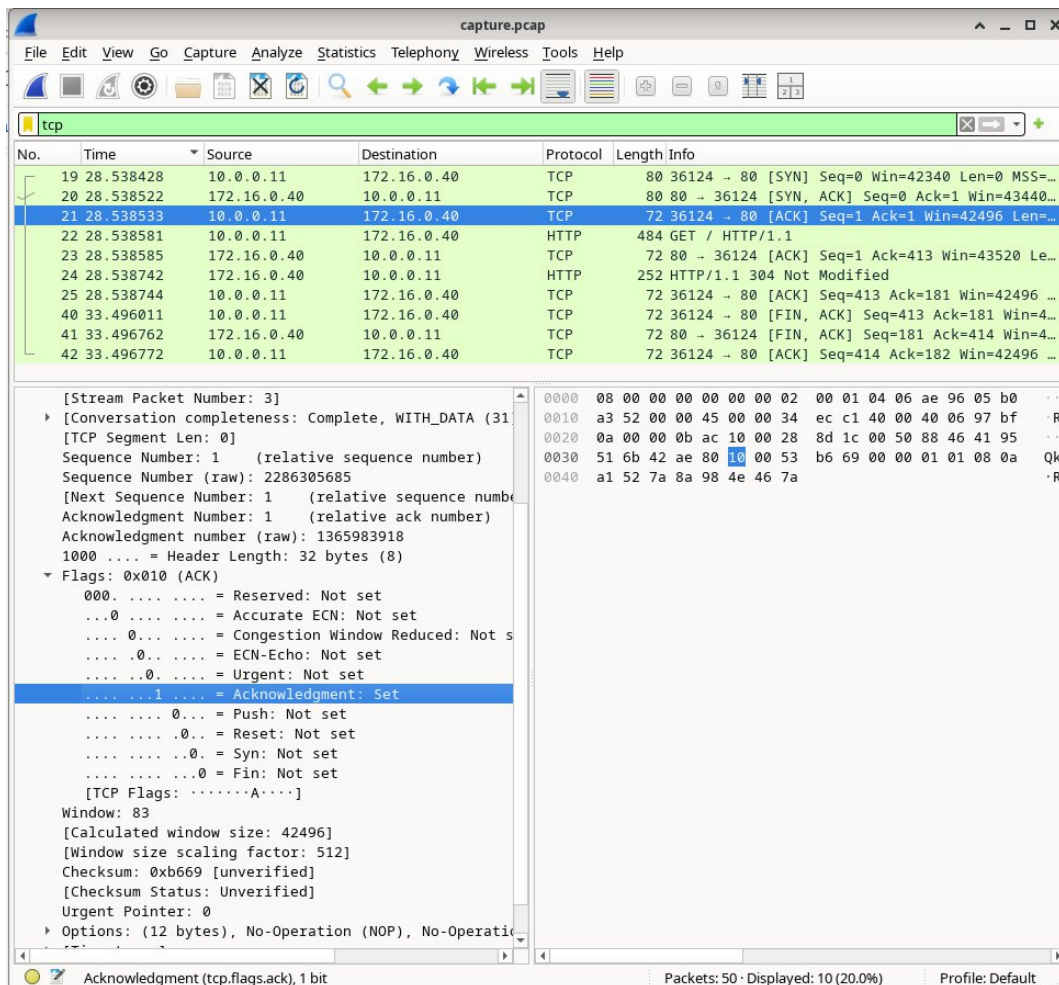
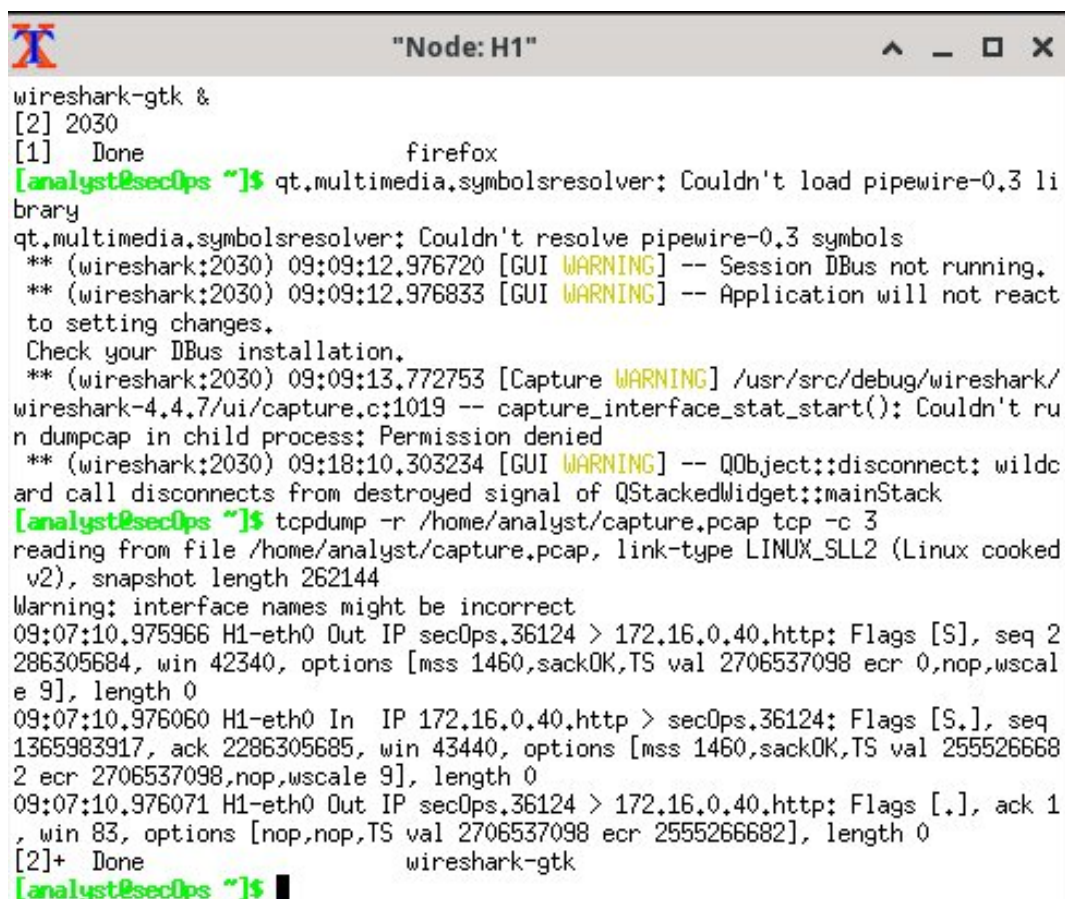


Figura 7 — Pacchetto ACK: flag 0x010, Acknowledgment: Set. Connessione stabilita.

Parte 3 — Visualizzazione con tcpdump

Nella terza parte è stato usato tcpdump da riga di comando per leggere il file pcap già salvato e visualizzare i primi 3 pacchetti TCP. Il comando usato è stato: `tcpdump -r /home/analyst/capture.pcap tcp -c 3`. L'output ha mostrato i tre pacchetti con la notazione [S] per il SYN, [S.] per il SYN-ACK e [.] per l'ACK.



```
Node: H1
wireshark-gtk &
[2] 2030
[1] Done firefox
[analyst@sec0ps ~]$ qt.multimedia.symbolsresolver: Couldn't load pipewire-0.3 library
qt.multimedia.symbolsresolver: Couldn't resolve pipewire-0.3 symbols
** (wireshark:2030) 09:09:12.976720 [GUI WARNING] -- Session DBus not running.
** (wireshark:2030) 09:09:12.976833 [GUI WARNING] -- Application will not react to setting changes.
Check your DBus installation.
** (wireshark:2030) 09:09:13.772753 [Capture WARNING] /usr/src/debug/wireshark/wireshark-4.4.7/ui/capture.c:1019 -- capture_interface_stat_start(): Couldn't run dumpcap in child process: Permission denied
** (wireshark:2030) 09:18:10.303234 [GUI WARNING] -- QObject::disconnect: wildcard call disconnects from destroyed signal of QStackedWidget::mainStack
[analyst@sec0ps ~]$ tcpdump -r /home/analyst/capture.pcap tcp -c 3
reading from file /home/analyst/capture.pcap, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
Warning: interface names might be incorrect
09:07:10.975966 H1-eth0 Out IP sec0ps.36124 > 172.16.0.40.http: Flags [S], seq 286305684, win 42340, options [mss 1460,sackOK,TS val 2706537098 ecr 0,nop,wscale 9], length 0
09:07:10.976060 H1-eth0 In IP 172.16.0.40.http > sec0ps.36124: Flags [S.], seq 1365983917, ack 2286305685, win 43440, options [mss 1460,sackOK,TS val 2555266682 ecr 2706537098,nop,wscale 9], length 0
09:07:10.976071 H1-eth0 Out IP sec0ps.36124 > 172.16.0.40.http: Flags [.], ack 1, win 83, options [nop,nop,TS val 2706537098 ecr 2555266682], length 0
[2]+ Done wireshark-gtk
[analyst@sec0ps ~]$
```

Figura 8 — Output tcpdump con i 3 pacchetti dell'handshake.

5. Risultati Osservati

L'analisi ha permesso di osservare in modo concreto come TCP stabilisce una connessione. I tre pacchetti dell'handshake sono stati identificati sia visivamente in Wireshark sia testualmente tramite tcpdump. I flag TCP corrispondevano esattamente a quanto previsto dalla teoria: SYN nel primo pacchetto, SYN+ACK nel secondo e solo ACK nel terzo. I numeri di sequenza e acknowledgment risultavano coerenti con il meccanismo di sincronizzazione TCP.

6. Conclusioni e Raccomandazioni

L'esercitazione ha confermato il funzionamento del protocollo TCP e l'utilità di strumenti come Wireshark e tcpdump nell'analisi del traffico di rete. Essere in grado di riconoscere l'handshake a 3 vie è una competenza fondamentale per un analista di sicurezza, poiché permette di distinguere connessioni legittime da comportamenti anomali, come scansioni SYN o tentativi di connessione non completati.

Domande di Riflessione

1. Tre filtri Wireshark utili per un amministratore di rete potrebbero essere: il filtro tcp per isolare solo il traffico TCP, il filtro http per analizzare le richieste web, e il filtro ip.addr == [indirizzo] per monitorare il traffico di un host specifico.

2. In una rete di produzione Wireshark potrebbe essere utilizzato per individuare traffico anomalo o non autorizzato, per diagnosticare problemi di connettività analizzando i pacchetti scambiati, e per verificare che le comunicazioni avvengano sui protocolli e le porte corrette.